

Practical: 2

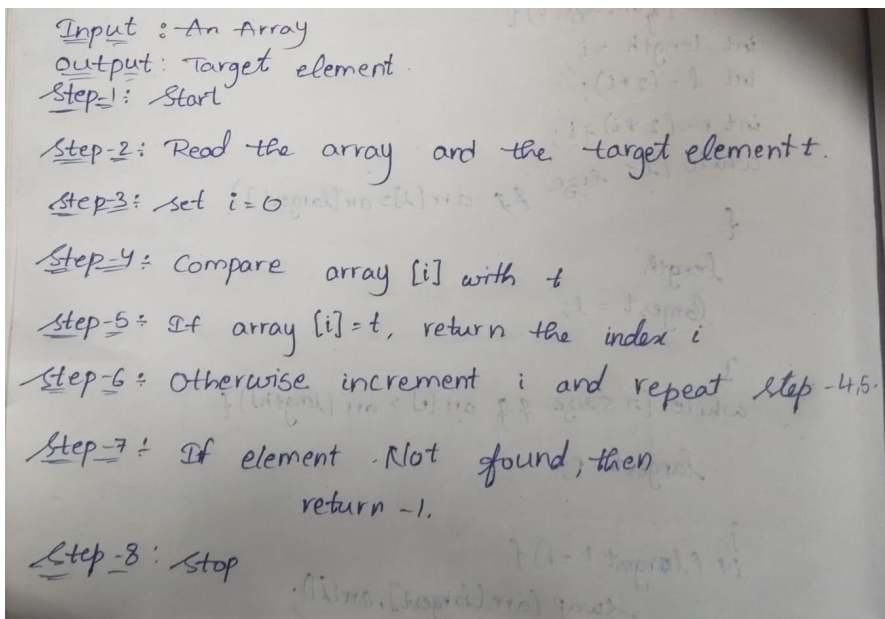
Implementation and Time analysis of linear and binary search algorithm

05/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:



Input :- An Array
Output :- Target element
Step-1 :- Start
Step-2 :- Read the array and the target element t .
Step-3 :- Set $i = 0$
Step-4 :- Compare array $[i]$ with t
Step-5 :- If array $[i] = t$, return the index i
Step-6 :- Otherwise increment i and repeat step -4,5.
Step-7 :- If element Not found, then
return -1.
Step-8 :- Stop

Program:

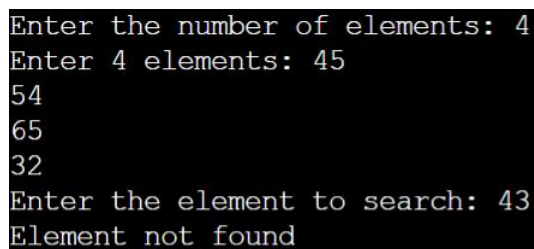
```
#include <iostream>
using namespace std;
int main() {    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    int target;    cout << "Enter the element
to search: ";
    cin >> target;
```

```
int index = -1;
for (int i = 0; i < n; i++) {
    if (arr[i] == target) {
        index = i;
        break;
    }
}

if (index != -1) {    cout << "Element found at index:
" << index << endl;
} else {
    cout << "Element not found" << endl;
}

return 0;
}
```

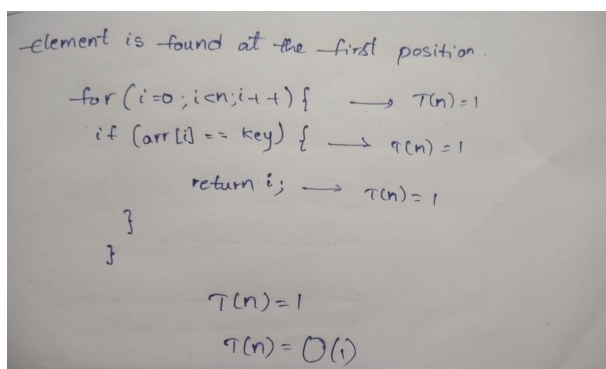
Output:



```
Enter the number of elements: 4
Enter 4 elements: 45
54
65
32
Enter the element to search: 43
Element not found
```

Time Complexity:

Best case:

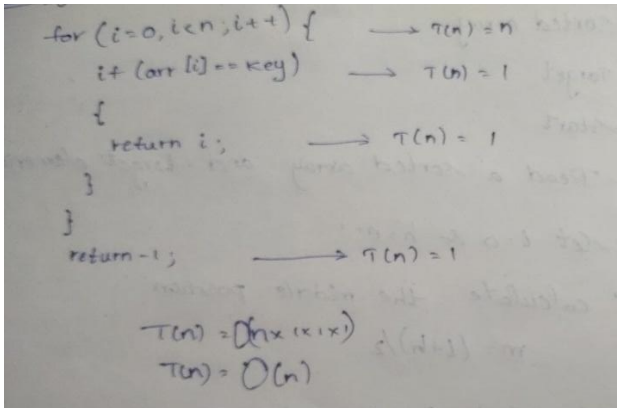


element is found at the first position.

```
for (i=0; i<n; i++) {    → T(n)=1
    if (arr[i] == key) { → T(n)=1
        return i;    → T(n)=1
    }
}
```

$T(n) = 1$
 $T(n) = O(1)$

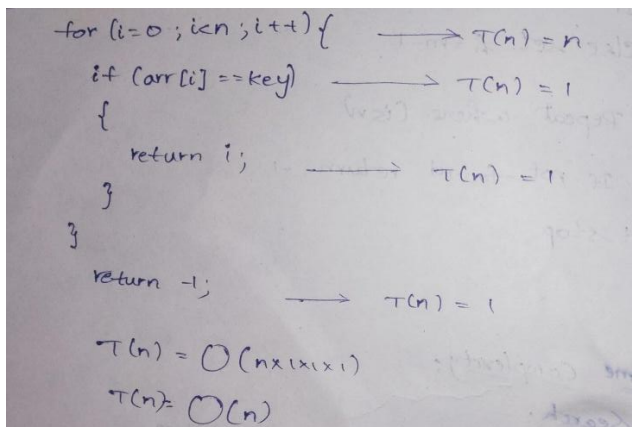
Average case: Element is somewhere in the middle



```
for (i=0; i<n; i++) {  
    if (arr[i] == key) {  
        return i;  
    }  
}  
return -1;
```

$T(n) = n$
 $T(n) = 1$
 $T(n) = 1$
 $T(n) = 1$
 $T(n) = O(n \times 1 \times 1)$
 $T(n) = O(n)$

Worst case: Element is at the last position / not found



```
for (i=0; i<n; i++) {  
    if (arr[i] == key) {  
        return i;  
    }  
}  
return -1;
```

$T(n) = n$
 $T(n) = 1$
 $T(n) = 1$
 $T(n) = 1$
 $T(n) = O(n \times 1 \times 1)$
 $T(n) = O(n)$

Conclusion:

Hence From this practical of linear search, the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.

Binary search

Algorithm:



Input: sorted array
output: target
Step-1: start
Step-2: Read a sorted array and target element.
Step-3: Set $l=0$ & $h=n-1$
Step-4: calculate the middle position
$$m = (l+h)/2$$

Step-5: If $arr[m] = t$ then return m
Step-6: If $arr[m] < t$, set $l = m+1$;
else set $h = m-1$
Step-7: Repeat where $(l \leq h)$
Step-8: If not found return -1
Step-9: stop

Program code:

```
#include <iostream>
#include <vector>
using namespace std;
int binarySearch(const vector<int>& arr, int target)
{
    int low = 0;    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
        {
            return mid;
        } else if (arr[mid] < target)
        {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
{
```

```
int size;    cout << "Enter the number of
elements: ";
cin >> size;

vector<int> arr(size);    cout << "Enter " << size << "
elements in sorted order:\n";
for (int i = 0; i < size; i++)
{
    cin >> arr[i];
}

int target;    cout << "Enter the element to
search for: ";
cin >> target;

int result = binarySearch(arr, target);

if (result != -1) {    cout << "Element found at index:
" << result << endl;
} else {
    cout << "Element not found in the array." << endl;
}

return 0;
}
```

Output:

```
Enter the number of elements: 4
Enter 4 elements in sorted order:
34
45
56
21
Enter the element to search for: 21
Element not found in the array.
```

+

Time complexity:

Best case:

Best case:
 $mid = (low + high) / 2$
 if $arr[mid] == key$
 return mid
 $T(n) = 1$
 $T(n) = O(1)$

Average case: Key is found after several divisions

```
low = 0; → T(n) = 1
high = n-1; → T(n) = 1
while (low <= high) { → T(n) = O(log n)
    mid = (low + high) / 2; → T(n) = 1
    if (arr[mid] == key) { → T(n) = 1
        return mid; → T(n) = 1
    }
    else if (arr[mid] < key) → T(n) = 1
    {
        low = mid + 1; } → T(n) = 1
    else {
        high = mid - 1; → T(n) = 1
    }
}
return -1; → T(n) = 1

T(n) = O(log n)
```

Worst case: Key is found at the last division / not found

```
low = 0; → T(n) = 1
high = n-1; → T(n) = 1
while (low <= high) { → T(n) = O(log n)
    mid = (low + high) / 2; → T(n) = 1
    if (arr[mid] == key) { → T(n) = 1
        return mid; → T(n) = 1
    }
    else if (arr[mid] < key) → T(n) = 1
    {
        low = mid + 1; } → T(n) = 1
    else {
        high = mid - 1; → T(n) = 1
    }
}
return -1; → T(n) = 1

T(n) = O(log n)
```

Conclusion:

From this practical of the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.

